Module 03 Lab Report - English

Comprehensive documentation of the Linux Security, Hardening, Monitoring, and Incident Management lab performed on an AWS EC2 Ubuntu 24.04 LTS instance.

Scope. This document covers the complete Module 03 lab flow from initial host inspection to SSH monitoring, Fail2ban deployment, manual validation, incident simulation, backdoor detection, containment, eradication, auditd deployment, audit rule creation, log interpretation, terminal usability fixes, and firewall hardening with UFW.

1. Lab Environment and Objectives

1.1 Environment

- Platform: AWS EC2
- OS: Ubuntu 24.04 LTS
- Hostname: `ip-172-31-42-76`
- Main user: `ubuntu`
- Key security goal: build a hardened and observable Linux server aligned with Module 03 and the Road Map project “Hardened Linux Server and Security Monitoring Lab”.

1.2 Learning Objectives

- Verify host integrity and obvious persistence mechanisms.
- Review SSH exposure and authentication artifacts.
- Deploy and harden Fail2ban against SSH abuse.
- Simulate an attacker-created account and privilege escalation path.
- Perform containment, eradication, and validation.
- Install auditd and create kernel-level file monitoring rules.
- Interpret audit logs technically, including `auid`, `exe`, `cwd`, and `syscall`.
- Apply host firewall policy with UFW using a default-deny model.

2. Initial Host Review and Security Baseline

2.1 Authorized SSH Key Inspection

The authorized SSH keys were inspected to determine whether an attacker had left persistence through an added public key. Only one key was present and it matched the expected EC2 key pair label.

```
ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAABAQCzwX6ufr0McWpAuZtL0YXnChEryGCKvI8QGi7ANHzttOhjzqpvDmjwgWjLjDCW1JeV
U/CRB8oTtO5/R5/3ZKn841Jf/
26dd8AQzTXJI0MNySebtM4q41yjXbsOlTOEvQF1lBlV9JZapZkNWw7zZ ozan-linux-lab-key1
```

Question	Finding
How many keys existed?	One
Key label	`ozan-linux-lab-key1`
Assessment	No unauthorized SSH key persistence observed

2.2 SSH Log Review

SSH logs for the current day were reviewed to identify successful logins, failed authentication attempts, and reconnaissance behavior.

```
sudo journalctl -u ssh --since "today" --no-pager | grep -E "Accepted|Invalid
user|Failed password|Connection closed by|Connection reset"
```

```
Aug 10 06:32:30 ip-172-31-42-76 sshd[970]: Accepted publickey for ubuntu from
212.252.96.252 port 10002 ssh2: RSA
SHA256:vOaaCSxjGEGP0zYQri3rtl5sYp6Jl5CVGpeCHJdX3Ek
Aug 10 06:35:43 ip-172-31-42-76 sshd[1136]: Connection closed by 154.0.185.10
port 10013
Aug 10 06:36:43 ip-172-31-42-76 sshd[1139]: Connection closed by 144.124.192.120
port 49719
Aug 10 06:37:44 ip-172-31-42-76 sshd[1763]: Connection closed by 181.85.208.53
port 44744
Aug 10 06:37:47 ip-172-31-42-76 sshd[1899]: Connection closed by 181.85.208.53
port 43015
Aug 10 07:24:07 ip-172-31-42-76 sshd[29580]: Connection closed by 212.47.142.107
port 10389
Aug 10 07:24:52 ip-172-31-42-76 sshd[29581]: Connection closed by 212.47.142.107
port 9291
Aug 10 07:58:46 ip-172-31-42-76 sshd[29643]: Connection closed by 119.148.49.82
port 6720
```

Assessment. The single successful login was a valid public-key login for user `ubuntu` from IP `212.252.96.252`. The remaining entries showed connection closures without a successful login. This indicated background SSH exposure and likely scanning activity, not confirmed compromise.

2.3 Source Counting by IP

The next step was to quantify source frequency.

```
sudo journalctl -u ssh --since "today" --no-pager | grep -E "Invalid user|Failed password|Connection closed by" | grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}' | sort | uniq -c | sort -rn
```

```
      2 212.47.142.107
      2 181.85.208.53
      1 154.0.185.10
      1 144.124.192.120
      1 119.148.49.82
```

This confirmed low-volume reconnaissance from five distinct IPs. At this point the host showed internet background noise but not high-intensity brute-force activity.

3. Deploying and Hardening Fail2ban

3.1 Installation

```
sudo apt update && sudo apt install fail2ban -y
```

Fail2ban and its dependencies were installed successfully. Package output showed installation of `fail2ban`, `python3-pyasyncore`, `python3-pyinotify`, and `whois`.

3.2 Initial Service Status

```
sudo fail2ban-client status
```

```
Status
|- Number of jail:      1
`- Jail list:   sshd
```

```
sudo fail2ban-client status sshd
```

```
Status for the jail: sshd
|- Filter
|  |- Currently failed: 0
|  |- Total failed:     1
|  `- Journal matches:  _SYSTEMD_UNIT=sshd.service + _COMM=sshd
`- Actions
   |- Currently banned: 0
   |- Total banned:     0
   `- Banned IP list:
```

The `sshd` jail was active, one failure had been observed, and no ban had yet been issued.

3.3 Default Threshold Analysis

```
sudo fail2ban-client get sshd maxretry
sudo fail2ban-client get sshd findtime
sudo fail2ban-client get sshd bantime
```

```
5
600
600
```

Parameter	Value	Meaning
`maxretry`	5	Five matching failures before a ban
`findtime`	600	Those failures must happen within 10 minutes
`bantime`	600	Ban duration was 10 minutes

The lab conclusion was that these defaults were functional but not strict enough for an internet-exposed SSH service.

3.4 Reviewing Fail2ban Log Activity

```
sudo tail -n 30 /var/log/fail2ban.log
```

```
2026-08-10 08:26:31,420 fail2ban.server         [30222]: INFO
--------------------------------------------------
2026-08-10 08:26:31,421 fail2ban.server         [30222]: INFO    Starting
Fail2ban v1.0.2
2026-08-10 08:26:31,421 fail2ban.observer       [30222]: INFO    Observer
start...
2026-08-10 08:26:31,428 fail2ban.database       [30222]: INFO    Connected to
fail2ban persistent database '/var/lib/fail2ban/fail2ban.sqlite3'
2026-08-10 08:26:31,431 fail2ban.database       [30222]: WARNING New database
created. Version '4'
2026-08-10 08:26:31,431 fail2ban.jail           [30222]: INFO    Creating new
jail 'sshd'
2026-08-10 08:26:31,674 fail2ban.jail           [30222]: INFO    Jail 'sshd'
uses systemd {}
2026-08-10 08:26:31,676 fail2ban.jail           [30222]: INFO    Initiated
'systemd' backend
2026-08-10 08:26:31,676 fail2ban.filter         [30222]: INFO      maxLines: 1
2026-08-10 08:26:31,693 fail2ban.filtersystemd  [30222]: INFO    [sshd] Added
journal match for: '_SYSTEMD_UNIT=sshd.service + _COMM=sshd'
2026-08-10 08:26:31,694 fail2ban.filter         [30222]: INFO      maxRetry: 5
2026-08-10 08:26:31,694 fail2ban.filter         [30222]: INFO      findtime: 600
2026-08-10 08:26:31,694 fail2ban.actions        [30222]: INFO      banTime: 600
2026-08-10 08:26:31,694 fail2ban.filter         [30222]: INFO      encoding:
UTF-8
2026-08-10 08:26:31,695 fail2ban.jail           [30222]: INFO    Jail 'sshd'
```

```
started
2026-08-10 08:26:31,700 fail2ban.filtersystemd  [30222]: INFO    [sshd] Jail is
in operation now (process new journal entries)
2026-08-10 08:37:35,131 fail2ban.filter         [30222]: INFO    [sshd] Found
185.200.36.42 - 2026-08-10 08:37:34
```

This proved the jail was live and already identifying suspicious source IPs.

3.5 Hardening Configuration via `jail.local`

A custom override file was created so that future package updates would not overwrite the local tuning.

```
[sshd]
enabled = true
maxretry = 3
findtime = 600
bantime = 3600
```

```
sudo systemctl restart fail2ban
sudo fail2ban-client get sshd maxretry
sudo fail2ban-client get sshd bantime
```

```
2026-08-10 09:53:04,025 fail2ban                [31161]: ERROR   Failed to
access socket path: /var/run/fail2ban/fail2ban.sock. Is fail2ban running?
3600
```

The first immediate query raced the service restart and briefly hit the socket-not-ready condition. A subsequent validation confirmed the service was healthy and the new values were loaded.

```
sudo systemctl status fail2ban --no-pager
sudo fail2ban-client get sshd maxretry
sudo fail2ban-client get sshd bantime
```

```
● fail2ban.service - Fail2Ban Service
     Loaded: loaded (/usr/lib/systemd/system/fail2ban.service; enabled; preset:
enabled)
     Active: active (running) since Mon 2026-08-10 09:53:03 UTC; 1min 42s ago
       Docs: man:fail2ban(1)
   Main PID: 31158 (fail2ban-server)
      Tasks: 5 (limit: 1013)
     Memory: 19.3M (peak: 19.3M)
        CPU: 364ms
     CGroup: /system.slice/fail2ban.service
             └─31158 /usr/bin/python3 /usr/bin/fail2ban-server -xf start

Aug 10 09:53:03 ip-172-31-42-76 systemd[1]: Started fail2ban.service - Fail2Ban
Service.
Aug 10 09:53:04 ip-172-31-42-76 fail2ban-server[31158]: 2026-08-10 09:53:04,022
```

```
fail2ban.configreader   [31158]: WARNING 'allowipv6' not defined in
'Definition'. U… one: 'auto'
Aug 10 09:53:04 ip-172-31-42-76 fail2ban-server[31158]: Server ready
3
3600
```

The hardened settings were now in effect: three failures within ten minutes would trigger a one-hour ban.

4. Validating the Ban Mechanism

4.1 Verifying Current State

```
sudo systemctl status fail2ban --no-pager && sudo fail2ban-client status sshd
```

```
● fail2ban.service - Fail2Ban Service
     Loaded: loaded (/usr/lib/systemd/system/fail2ban.service; enabled; preset:
enabled)
     Active: active (running) since Mon 2026-08-10 09:53:03 UTC; 13min ago
...
Status for the jail: sshd
|- Filter
|  |- Currently failed: 0
|  |- Total failed:     0
|  `- Journal matches:  _SYSTEMD_UNIT=sshd.service + _COMM=sshd
`- Actions
   |- Currently banned: 0
   |- Total banned:     0
   `- Banned IP list:
```

4.2 Identifying Active User IPs Before Testing

```
echo "Benim IP'm:" && who | awk '{print $5}' | tr -d '()'
```

```
Benim IP'm:
212.252.96.252
3.120.181.44
```

This step was used to avoid accidentally banning an active administrative source.

4.3 Manual Ban Test

Because there was no safe second machine available for a real repeated-failure test, the action backend itself was validated manually.

```
sudo fail2ban-client set sshd banip 1.1.1.1
```

```
1
```

```
sudo fail2ban-client status sshd
```

```
Status for the jail: sshd
|- Filter
|  |- Currently failed: 0
|  |- Total failed:     0
|  `- Journal matches:  _SYSTEMD_UNIT=sshd.service + _COMM=sshd
`- Actions
   |- Currently banned: 1
   |- Total banned:     1
   `- Banned IP list:   1.1.1.1
```

```
sudo tail -n 5 /var/log/fail2ban.log
```

```
2026-08-10 09:53:04,212 fail2ban.actions        [31158]: INFO      banTime: 3600
2026-08-10 09:53:04,212 fail2ban.filter         [31158]: INFO      encoding:
UTF-8
2026-08-10 09:53:04,213 fail2ban.jail           [31158]: INFO    Jail 'sshd'
started
2026-08-10 09:53:04,214 fail2ban.filtersystemd  [31158]: INFO    [sshd] Jail is
in operation now (process new journal entries)
2026-08-10 10:07:30,751 fail2ban.actions        [31158]: NOTICE  [sshd] Ban
1.1.1.1
```

This conclusively proved the ban action path was operational.

4.4 Manual Unban Test

```
sudo fail2ban-client set sshd unbanip 1.1.1.1
```

```
0
```

```
sudo fail2ban-client status sshd
```

```
Status for the jail: sshd
|- Filter
|  |- Currently failed: 0
|  |- Total failed:     0
|  `- Journal matches:  _SYSTEMD_UNIT=sshd.service + _COMM=sshd
`- Actions
   |- Currently banned: 0
   |- Total banned:     1
   `- Banned IP list:
```

The IP was successfully removed from the active ban list, while the historical total-banned counter remained as evidence of the test event.

5. Incident Simulation - Unauthorized User and Hidden Sudo Backdoor

5.1 Simulating an Attacker-Created User

An attacker-style user was deliberately created to practice triage and incident response workflows.

```
sudo adduser sys-update
```

```
info: Adding user `sys-update' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `sys-update' (1001) ...
info: Adding new user `sys-update' (1001) with group `sys-update (1001)' ...
info: Creating home directory `/home/sys-update' ...
info: Copying files from `/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for sys-update
Enter the new value, or press ENTER for the default
        Full Name []: bilgi_islem
        Room Number []: 511
        Work Phone []:
        Home Phone []:
        Other []:
Is the information correct? [Y/n] y
info: Adding new user `sys-update' to supplemental / extra groups `users' ...
info: Adding user `sys-update' to group `users' ...
```

5.2 Triage Through `/etc/passwd`

```
tail -n 5 /etc/passwd
```

```
polkitd:x:989:989:User for polkitd:/:/usr/sbin/nologin
ec2-instance-connect:x:109:65534::/nonexistent:/usr/sbin/nologin
_chrony:x:110:112:Chrony daemon,,,:/var/lib/chrony:/usr/sbin/nologin
ubuntu:x:1000:1000:Ubuntu:/home/ubuntu:/bin/bash
sys-update:x:1001:1001:bilgi_islem,511,,:/home/sys-update:/bin/bash
```

```
grep -E '/(bash|sh|zsh)$' /etc/passwd
```

```
root:x:0:0:root:/root:/bin/bash
ubuntu:x:1000:1000:Ubuntu:/home/ubuntu:/bin/bash
sys-update:x:1001:1001:bilgi_islem,511,,:/home/sys-update:/bin/bash
```

Why it was suspicious. The name `sys-update` looked service-like, but the account had a human UID range, a real home directory, and a full login shell. This was a red flag.

5.3 Checking Sudo Privilege Paths

```
getent group sudo
```

```
sudo:x:27:ubuntu
```

```
sudo grep -r "sys-update" /etc/sudoers /etc/sudoers.d/
```

No direct match was initially found. This meant the new user was not yet obviously privileged.

5.4 Simulating Hidden Sudo Persistence

The next stage simulated an attacker adding a stealthy privilege escalation path through a custom sudoers drop-in.

```
echo "sys-update ALL=(ALL) NOPASSWD:ALL" | sudo tee /etc/sudoers.d/sys-logs
```

```
sys-update ALL=(ALL) NOPASSWD:ALL
```

```
sudo sudo -l -U sys-update
```

```
Matching Defaults entries for sys-update on ip-172-31-42-76:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/
sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty

User sys-update may run the following commands on ip-172-31-42-76:
    (ALL) NOPASSWD: ALL
```

This was the core incident finding: the account had become a passwordless sudo-capable root path.

5.5 Reviewing the Sudoers Directory

```
ls -l /etc/sudoers.d/
```

```
ls: cannot open directory '/etc/sudoers.d/': Permission denied
```

This failure reinforced an operational lesson: highly sensitive directories require elevated privilege even for listing.

```
sudo ls -l /etc/sudoers.d/
```

```
total 12
-r--r----- 1 root root  139 Aug  7 06:42 90-cloud-init-users
-r--r----- 1 root root 1068 Jan 29  2024 README
-rw-r--r-- 1 root root   34 Aug 10 18:34 sys-logs
```

The malicious-looking drop-in file `sys-logs` was visible and attributable.

6. Containment, Eradication, and Validation

6.1 Removing the Backdoor File

```
sudo rm /etc/sudoers.d/sys-logs
```

6.2 Locking the Account

```
sudo usermod _l sys-update
```

```
Usage: usermod [options] LOGIN
...
  -L, --lock                    lock the user account
...
```

The first attempt used `_l` instead of `-L`. This produced usage output and was corrected.

```
sudo usermod -L sys-update
```

6.3 Terminating Active Processes

```
sudo pkill -u sys-update
```

6.4 Validating Loss of Sudo Capability

```
sudo sudo -l -U sys-update
```

```
User sys-update is not allowed to run sudo on ip-172-31-42-76.
```

At this point the privileged persistence mechanism had been neutralized.

7. Post-Incident Forensics

7.1 Home Directory Review

```
sudo ls -la /home/sys-update/
```

```
total 20
drwxr-x--- 2 sys-update sys-update 4096 Aug 10 13:30 .
drwxr-xr-x 4 root       root       4096 Aug 10 13:30 ..
-rw-r--r-- 1 sys-update sys-update  220 Aug 10 13:30 .bash_logout
-rw-r--r-- 1 sys-update sys-update 3771 Aug 10 13:30 .bashrc
-rw-r--r-- 1 sys-update sys-update  807 Aug 10 13:30 .profile
```

Only standard skeleton files existed.

7.2 Shell History Review

```
sudo cat /home/sys-update/.bash_history 2>/dev/null || echo "Bash history
dosyasi bulunamadi"
```

```
Bash history dosyasi bulunamadi
```

No interactive command history was present for the account.

7.3 Timeline Reconstruction from Logs

```
sudo grep -r "sys-update" /var/log/auth.log 2>/dev/null || sudo journalctl --
since "today" --no-pager | grep -E "sys-update|useradd|adduser"
```

```
2026-08-10T13:30:07.815025+00:00 ip-172-31-42-76 sudo:   ubuntu : TTY=pts/
0 ; PWD=/home/ubuntu ; USER=root ; COMMAND=/usr/sbin/adduser sys-update
2026-08-10T13:30:07.899622+00:00 ip-172-31-42-76 groupadd[32886]: group
added to /etc/group: name=sys-update, GID=1001
2026-08-10T13:30:07.906111+00:00 ip-172-31-42-76 groupadd[32886]: group
added to /etc/gshadow: name=sys-update
2026-08-10T13:30:07.907620+00:00 ip-172-31-42-76 groupadd[32886]: new group:
name=sys-update, GID=1001
2026-08-10T13:30:07.945706+00:00 ip-172-31-42-76 useradd[32893]: new user:
name=sys-update, UID=1001, GID=1001, home=/home/sys-update, shell=/bin/bash,
from=/dev/pts/1
2026-08-10T13:30:16.069574+00:00 ip-172-31-42-76 passwd[32906]:
pam_unix(passwd:chauthtok): password changed for sys-update
2026-08-10T13:30:40.089277+00:00 ip-172-31-42-76 chfn[32910]: changed user
'sys-update' information
2026-08-10T13:30:42.172864+00:00 ip-172-31-42-76 gpasswd[32920]: members of
group users set by root to sys-update
2026-08-10T18:29:37.239572+00:00 ip-172-31-42-76 sudo:   ubuntu : TTY=pts/
0 ; PWD=/home/ubuntu ; USER=root ; COMMAND=/usr/bin/grep -r sys-update /etc/
```

```
sudoers /etc/sudoers.d/
2026-08-10T18:29:48.721009+00:00 ip-172-31-42-76 sudo:   ubuntu : TTY=pts/
0 ; PWD=/home/ubuntu ; USER=root ; COMMAND=/usr/bin/grep -r sys-update /etc/
sudoers /etc/sudoers.d/
2026-08-10T18:34:57.321544+00:00 ip-172-31-42-76 sudo:   ubuntu : TTY=pts/
0 ; PWD=/home/ubuntu ; USER=root ; COMMAND=/usr/bin/sudo -l -U sys-update
2026-08-10T18:59:53.968682+00:00 ip-172-31-42-76 sudo:   ubuntu : TTY=pts/
0 ; PWD=/home/ubuntu ; USER=root ; COMMAND=/usr/sbin/usermod _l sys-update
2026-08-10T19:00:10.179083+00:00 ip-172-31-42-76 sudo:   ubuntu : TTY=pts/
0 ; PWD=/home/ubuntu ; USER=root ; COMMAND=/usr/sbin/usermod -L sys-update
2026-08-10T19:00:10.191856+00:00 ip-172-31-42-76 usermod[34632]: lock user
'sys-update' password
2026-08-10T19:00:26.845724+00:00 ip-172-31-42-76 sudo:   ubuntu : TTY=pts/
0 ; PWD=/home/ubuntu ; USER=root ; COMMAND=/usr/bin/pkill -u sys-update
2026-08-10T19:00:42.959731+00:00 ip-172-31-42-76 sudo:   ubuntu : TTY=pts/
0 ; PWD=/home/ubuntu ; USER=root ; COMMAND=/usr/bin/sudo -l -U sys-update
2026-08-10T19:06:41.707346+00:00 ip-172-31-42-76 sudo:   ubuntu : TTY=pts/
0 ; PWD=/home/ubuntu ; USER=root ; COMMAND=/usr/bin/ls -la /home/sys-update/
2026-08-10T19:06:55.239179+00:00 ip-172-31-42-76 sudo:   ubuntu : TTY=pts/
0 ; PWD=/home/ubuntu ; USER=root ; COMMAND=/usr/bin/cat /home/sys-
update/.bash_history
2026-08-10T19:07:06.496661+00:00 ip-172-31-42-76 sudo:   ubuntu : TTY=pts/
0 ; PWD=/home/ubuntu ; USER=root ; COMMAND=/usr/bin/grep -r sys-update /var/
log/auth.log
```

This provided a precise timeline of user creation, metadata modification, containment actions, and forensic queries.

7.4 Checking for Audit Framework Availability Before Installation

```
sudo auditctl -s 2>/dev/null || echo "auditctl bulunamadi"
```

```
auditctl bulunamadi
```

This confirmed that kernel auditing had not yet been deployed at that stage.

7.5 Removing the Simulated User

```
sudo userdel -r sys-update
```

```
userdel: sys-update mail spool (/var/mail/sys-update) not found
```

The warning was normal because the user had no mail spool file.

```
getent passwd sys-update || echo "sys-update kullanicisi silindi"
sudo ls -ld /home/sys-update 2>/dev/null || echo "sys-update home dizini
silindi"
```

```
sudo grep -R "sys-update" /etc/sudoers /etc/sudoers.d/ || echo "sudoers icinde
sys-update yetkisi yok"
```

```
sys-update kullanicisi silindi
sys-update home dizini silindi
sudoers icinde sys-update yetkisi yok
```

The account, home directory, and sudoers reference were fully removed.

8. Deploying auditd for Kernel-Level Monitoring

8.1 Rationale

Traditional service logs explain what services recorded. They do not always show low-level file and syscall behavior with the same reliability or granularity. The Linux audit framework fills this gap by recording kernel-mediated security-relevant events.

8.2 Installation and Service Validation

```
sudo apt update && sudo apt install auditd audispd-plugins -y
```

```
sudo systemctl status auditd
```

```
● auditd.service - Security Auditing Service
     Loaded: loaded (/usr/lib/systemd/system/auditd.service; enabled; preset:
enabled)
     Active: active (running) since Tue 2026-08-11 10:31:29 UTC; 10s ago
       Docs: man:auditd(8)
             https://github.com/linux-audit/audit-documentation
    Process: 2129 ExecStart=/sbin/auditd (code=exited, status=0/SUCCESS)
    Process: 2134 ExecStartPost=/sbin/augenrules --load (code=exited, status=0/
SUCCESS)
   Main PID: 2130 (auditd)
      Tasks: 2 (limit: 627)
     Memory: 732.0K (peak: 2.5M)
        CPU: 29ms
     CGroup: /system.slice/auditd.service
             └─2130 /sbin/auditd

Aug 11 10:31:29 ip-172-31-42-76 augenrules[2144]: enabled 1
Aug 11 10:31:29 ip-172-31-42-76 augenrules[2144]: failure 1
Aug 11 10:31:29 ip-172-31-42-76 augenrules[2144]: pid 2130
Aug 11 10:31:29 ip-172-31-42-76 augenrules[2144]: rate_limit 0
Aug 11 10:31:29 ip-172-31-42-76 augenrules[2144]: backlog_limit 8192
Aug 11 10:31:29 ip-172-31-42-76 augenrules[2144]: lost 0
Aug 11 10:31:29 ip-172-31-42-76 augenrules[2144]: backlog 0
Aug 11 10:31:29 ip-172-31-42-76 augenrules[2144]: backlog_wait_time 60000
Aug 11 10:31:29 ip-172-31-42-76 augenrules[2144]: backlog_wait_time_actual 0
Aug 11 10:31:29 ip-172-31-42-76 systemd[1]: Started auditd.service - Security
Auditing Service.
```

Interpretation. The service was healthy. `failure 1` reflected the audit failure mode rather than a failed service start. `lost 0` and `backlog 0` showed no lost or queued audit events.

8.3 Confirming No Rules Were Loaded Yet

```
sudo auditctl -l
```

```
No rules
```

```
sudo ls -la /etc/audit/rules.d/
```

```
total 12
drwxr-x--- 2 root root 4096 Aug 11 10:31 .
drwxr-x--- 4 root root 4096 Aug 11 10:31 ..
-rw-r----- 1 root root  244 Oct  2  2024 audit.rules
```

The audit daemon was running, but no watch rules had been loaded yet.

8.4 Creating Persistent Audit Rules

A custom rules file was created to monitor user account changes and sudoers drop-in modifications.

```
-w /etc/passwd -p wa -k user_changes
-w /etc/sudoers.d/ -p wa -k sudoers_changes
```

```
sudo augenrules --load
sudo auditctl -l
```

```
No rules
enabled 1
failure 1
pid 2130
rate_limit 0
backlog_limit 8192
lost 0
backlog 0
backlog_wait_time 60000
backlog_wait_time_actual 0
enabled 1
failure 1
pid 2130
rate_limit 0
backlog_limit 8192
lost 0
backlog 0
backlog_wait_time 60000
backlog_wait_time_actual 0
enabled 1
failure 1
pid 2130
rate_limit 0
backlog_limit 8192
lost 0
backlog 0
backlog_wait_time 60000
backlog_wait_time_actual 0
```

```
-w /etc/passwd -p wa -k user_changes
-w /etc/sudoers.d -p wa -k sudoers_changes
```

The final lines confirmed that both watch rules were loaded into the kernel.

8.5 Validating the `user_changes` Rule

The lab triggered an attribute/write-related event against `/etc/passwd` and then reviewed the resulting audit records.

```
sudo ausearch -k user_changes
```

```
----
time->Tue Aug 11 10:41:29 2026
type=PROCTITLE msg=audit(1786444889.183:187):
proctitle=2F7362696E2F617564697463746C002D52002F6574632F61756469742F61756469742E72756C6573
type=SYSCALL msg=audit(1786444889.183:187): arch=c000003e syscall=44
success=yes exit=1080 a0=3 a1=7ffede479980 a2=438 a3=0 items=0 ppid=2285
pid=2299 auid=1000 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0
tty=pts1 ses=1 comm="auditctl" exe="/usr/sbin/auditctl" subj=unconfined
key=(null)
type=CONFIG_CHANGE msg=audit(1786444889.183:187): auid=1000 ses=1
subj=unconfined op=add_rule key="user_changes" list=4 res=1
----
time->Tue Aug 11 10:43:06 2026
type=PROCTITLE msg=audit(1786444986.374:202):
proctitle=746F756368002F6574632F706173737764
type=PATH msg=audit(1786444986.374:202): item=0 name="/etc/passwd"
inode=2937 dev=103:01 mode=0100644 ouid=0 ogid=0 rdev=00:00 nametype=NORMAL
cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
type=CWD msg=audit(1786444986.374:202): cwd="/home/ubuntu"
type=SYSCALL msg=audit(1786444986.374:202): arch=c000003e syscall=257
success=yes exit=3 a0=ffffff9c a1=7ffdd1edb7c5 a2=941 a3=1b6 items=1
ppid=2314 pid=2315 auid=1000 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0
fsgid=0 tty=pts1 ses=1 comm="touch" exe="/usr/bin/touch" subj=unconfined
key="user_changes"
```

Field	Observed value	Analytical meaning
`name`	`/etc/passwd`	The critical file that was touched
`exe`	`/usr/bin/touch`	The binary that performed the action
`cwd`	`/home/ubuntu`	Working directory at execution time
`auid`	1000	Original login identity behind the action; extremely valuable in forensics

`syscall`	257	Kernel syscall number, here corresponding to `openat`

This was the central auditd lesson of the lab: even when commands run with elevated privileges, the audit trail preserves the original accountable identity.

9. Terminal Readability Improvements

9.1 Initial Attempt and Failure

The terminal output was difficult to read. A first manual edit to `~/.bashrc` caused a syntax error.

```
source ~/.bashrc
```

```
-bash: /home/ubuntu/.bashrc: line 121: syntax error: unexpected end of file
```

A broken section of `.bashrc` showed that aliases and the `if ... fi` structure had been accidentally merged together.

9.2 Recovery

```
cp /etc/skel/.bashrc ~/
```

```
source ~/.bashrc
```

The clean skeleton copy restored a working shell configuration with no syntax error.

10. Firewall Hardening with UFW

10.1 Initial UFW State

```
sudo ufw status
```

```
Status: active

To                         Action      From
--                         ------      ----
22/tcp                     ALLOW       Anywhere
80/tcp                     ALLOW       Anywhere
22/tcp (v6)                ALLOW       Anywhere (v6)
80/tcp (v6)                ALLOW       Anywhere (v6)
```

SSH and HTTP were already allowed, and UFW was active.

10.2 Enforcing Default Policies

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
```

```
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
```

Meaning. All inbound traffic is denied unless explicitly allowed. Outbound traffic is allowed by default so the host can still reach the network for updates, package installation, and normal service operation.

10.3 Explicitly Allowing Required Services

```
sudo ufw allow ssh
```

```
Skipping adding existing rule
Skipping adding existing rule (v6)
```

```
sudo ufw allow http
```

```
Skipping adding existing rule
Skipping adding existing rule (v6)
```

These rules already existed, which is why UFW reported them as duplicates.

10.4 Enabling and Verifying Firewall Policy

```
sudo ufw enable
```

```
Command may disrupt existing ssh connections. Proceed with operation (y|n)? y
Firewall is active and enabled on system startup
```

```
sudo ufw status verbose
```

```
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp                     ALLOW IN    Anywhere
80/tcp                     ALLOW IN    Anywhere
```

```
22/tcp (v6)                ALLOW IN    Anywhere (v6)
80/tcp (v6)                ALLOW IN    Anywhere (v6)
```

The host firewall now matched a sound baseline: deny everything inbound except explicitly required services.

11. Technical Conclusions

11.1 What Was Accomplished

Domain	Outcome
SSH trust	Authorized key inspected; no unauthorized key observed
External exposure	Background SSH scanning identified and measured
Brute-force defense	Fail2ban installed, tuned, and action path validated
Incident response	Unauthorized user and sudo backdoor simulated, detected, contained, and removed
Forensics	Home directory and log timeline reviewed to reconstruct actions
Kernel auditing	auditd deployed and critical file watches loaded
Firewalling	UFW default-deny inbound policy confirmed with explicit allow rules

11.2 Threat - Control - Validation Summary

Threat	Control	Validation
Low-volume SSH reconnaissance	Fail2ban monitoring and hardened thresholds	Manual ban/unban test and log confirmation
Unauthorized account creation	Passwd review, sudoers inspection, lock and removal workflow	`sudo -l -U sys-update` changed from full sudo to no sudo
Silent modification of critical files	auditd watch rules	`ausearch -k user_changes` captured the event and accountable identity
Unnecessary inbound exposure	UFW default deny incoming	`sudo ufw status verbose`

11.3 Final Assessment

By the end of this lab, the host had evolved from a merely reachable Linux server into a substantially more mature system with layered protection and better visibility. Module 03 objectives were addressed not only conceptually but operationally: detection, hardening, containment, evidence review, and validation were all exercised on the live environment.

End of English report.